

6. Write a program to implement Naïve Bayes algorithm in python and to display the results using confusion matrix and accuracy.

AIM:

To implement the **Naïve Bayes algorithm** in Python and evaluate its performance using a **confusion matrix and accuracy**.

ALGORITHM:

- ☐ Import the required libraries.
- ☐ Create the dataset with input features and class labels.
- ☐ Split the dataset into training and testing data.
- ☐ Create a Naïve Bayes classifier.
- ☐ Train the classifier using the training data.
- ☐ Predict the class labels for the test data.
- ☐ Generate the confusion matrix.
- ☐ Calculate the accuracy of the classifier.
- ☐ Display the confusion matrix and accuracy.

CODE:

```
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, accuracy_score

X = [[1, 2], [2, 3], [3, 3], [4, 4],
      [6, 7], [7, 8], [8, 8], [9, 9]]
y = [0, 0, 0, 0, 1, 1, 1, 1]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)
model = GaussianNB()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
cm = confusion_matrix(y_test, y_pred)
accuracy = accuracy_score(y_test, y_pred)
print("Confusion Matrix:")
print(cm)
```

```
print("Accuracy:", accuracy * 100, "%")
```

OUTPUT:

```
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, accuracy_score

X = [[1, 2], [2, 3], [3, 3], [4, 4],
      [6, 7], [7, 8], [8, 8], [9, 9]]

y = [0, 0, 0, 0, 1, 1, 1, 1]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

model = GaussianNB()

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

cm = confusion_matrix(y_test, y_pred)
accuracy = accuracy_score(y_test, y_pred)

print("Confusion Matrix:")
print(cm)

print("Accuracy:", accuracy * 100, "%")
```

```
... Confusion Matrix:
[[1 0]
 [0 1]]
Accuracy: 100.0 %
```

Result Statement

Thus, the **Naïve Bayes algorithm** was successfully implemented in Python. The model performance was evaluated using a **confusion matrix**, and an accuracy of **100%** was obtained.